

## UNIT I

### 1. Discuss about Linear Algebra, probability & calculus?

#### Introduction to Linear Algebra

Linear algebra is a fundamental topic in the subject of mathematics and is extremely pervasive in the physical sciences. It also forms the backbone of many machine learning algorithms. Hence it is crucial for the deep learning practitioner to understand the core ideas.

Linear algebra is a branch of continuous, rather than discrete mathematics. The mathematician, physicist, engineer and quant will likely be familiar with continuous mathematics through the study of differential equations, which are used to model many physical and financial phenomena.

**Linear algebra, probability and calculus** are the 'languages' in which machine learning is written. Learning these topics will provide a deeper understanding of the underlying algorithmic mechanics and allow development of new algorithms, which can ultimately be deployed as more sophisticated quantitative trading strategies.

Many supervised **machine learning and deep learning** algorithms largely entail optimizing a loss function by adjusting model parameters. To carry this out requires some notion of how the loss function changes as the parameters of the model are varied.

This immediately motivates calculus—the elementary topic in mathematics which describes changes of quantities with respect to another. In particular it requires the concept of a partial derivative, which specifies how the **loss function** is altered through individual changes in each parameter.

These **partial derivatives** are often grouped together—in matrices—to allow more straightforward calculation. Even the most elementary machine learning models such as linear regression are optimized with these linear algebra techniques.

A key topic in linear algebra is that of **vector and matrix notation**. Being able to 'read the language' of linear algebra will open up the ability to understand textbooks, web posts and research papers that contain more complex model descriptions. This will not only allow reproduction and verification of existing models, but will allow extensions and new developments that can subsequently be deployed in trading strategies.

Linear algebra provides the first steps into **vectorization**, presenting a deeper way of thinking about parallelization of certain operations. Algorithms written in standard 'for-loop' notation can be reformulated as matrix equations providing significant gains in computational efficiency.

Such methods are used in the major Python libraries such as NumPy, SciPy, Scikit-Learn, Pandas and TensorFlow. GPUs have been designed to carry out optimized linear algebra operations. The explosive

growth in deep learning can partially be attributed to the highly parallelized nature of the underlying algorithms on commodity GPU hardware.

Linear algebra is a continuous mathematics subject but ultimately the entities such as **Scalars, Vectors, Tensors and Matrices** are implemented in a discrete computational environment. These discrete representations of linear algebra entities can lead to issues of overflow and underflow, which represent the limits of effectively representing extremely large and small numbers computationally.

One mechanism for mitigating the effects of limited numerical presentation is to make use of matrix factorization techniques. Such techniques allow certain matrices to be represented in terms of simpler, structured matrices that have useful computational properties.

Matrix decomposition techniques include **Lower Upper (LU)** decomposition, **QR decomposition** and **Singular Value Decomposition (SVD)**. They are an intrinsic component of certain machine learning algorithms including Linear Least Squares and **Principal Components Analysis (PCA)**. Matrix decomposition will be discussed at length later in this series.

## 2. Discuss about Scalars, Vectors, Matrices, and Tensors?

**Scalars, Vectors, Matrices, and Tensors:** These are the key elements of linear algebra.

**1. Scalars:** Scalars are single numerical values and are an example of a 0th-order (Zeroth Order) tensor that can be used to represent quantities such as temperature, time, or any other measurable quantity. They have magnitude but no direction. Examples of scalars include temperatures like 30 degrees Celsius or time intervals like 5 seconds. In practical terms, scalars often represent fundamental quantities such as constants, probabilities, or error metrics.

For instance, within Machine Learning, a scalar may denote the accuracy of a model or the value of a loss function. Similarly, in Data Science, scalars are employed to encapsulate statistical metrics like mean, variance, or correlation coefficients. Despite their apparent simplicity, scalars assume a critical role in various AI-ML-DS tasks, spanning optimization, regression analysis, and classification algorithms. Proficiency in understanding scalars forms the bedrock for comprehending more intricate concepts prevalent in these fields.

The notation  $\mathbf{x} \in \mathbf{R}$  states that the (lowercase) scalar value  $\mathbf{x}$  is an element of (or member of) the set of real-valued numbers,  $\mathbf{R}$ . There are various sets of numbers of interest within machine learning.  $\mathbf{N}$  represents the set of positive integers (1,2,3...).  $\mathbf{Z}$  represents the integers, which include positive, negative and zero values.  $\mathbf{Q}$  represents the set of *rational* numbers that may be expressed as a fraction of two integers.

**2. Vectors:** Vectors are ordered arrays of single numbers and are an example of 1st-order tensor. Vectors are members of objects known as vector spaces. A vector space can be thought of as the entire collection of *all* possible vectors of a particular length (or dimension). The three-dimensional real-valued vector

space, denoted by  $\mathbf{R}^3$  is often used to represent our real-world notion of three-dimensional space mathematically.

More formally a vector space is an  $n$ -dimensional **Cartesian product** of a set with itself, along with proper definitions on how to add vectors and multiply them with scalar values. If all of the scalars in a vector are real-valued then the notation  $\mathbf{x} \in \mathbf{R}^n$  states that the (boldface lowercase) vector value  $\mathbf{x}$  is a member of the  **$n$ -dimensional** vector space of real numbers  $\mathbf{R}^n$ .

Sometimes it is necessary to identify the *components* of a vector explicitly. The  **$i$ th** scalar element of a vector is written as  $x_i$ . Notice that this is non-bold lowercase since the element is a scalar. An  **$n$ -dimensional** vector itself can be explicitly written using the following notation:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

One of the primary use cases for vectors is to represent physical quantities that have both a *magnitude* and a *direction*. Scalars are only capable of representing magnitudes.

For instance scalars and vectors encode the difference between the *speed* of a car and its *velocity*. The velocity contains not only its speed but also its direction of travel. It is not difficult to imagine many more physical quantities that possess similar characteristics such as gravitational and electromagnetic forces or wind velocity.

In machine learning vectors often represent *feature vectors*, with their individual components specifying how important a particular feature is. Such features could include relative importance of words in a text document, the intensity of a set of pixels in a two-dimensional image or historical price values for a cross-section of financial instruments.

**3. Matrices:** Matrices are rectangular arrays consisting of numbers and are an example of 2nd-order tensors. If  $m$  and  $n$  are positive integers, that is  $m, n \in \mathbf{N}$  then the matrix contains  $mn$  numbers, with  $m$  rows and  $n$  columns.

If all of the scalars in a matrix are real-valued then a matrix is denoted with uppercase boldface letters, such as  $\mathbf{A} \in \mathbf{R}^{m \times n}$ . That is, the matrix lives in a  $m \times n$ -dimensional real-valued vector space. Hence matrices are really vectors that are just written in a two-dimensional table-like manner.

Its components are now identified by two indices  $i$  and  $j$ .  $i$  represents the index to the matrix row, while  $j$  represents the index to the matrix column. Each component of  $\mathbf{A}$  identified by  $a_{ij}$ . The full  $m \times n$  matrix can be written as:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \dots & a_{mn} \end{bmatrix}$$

It is often useful to abbreviate the full matrix component display into the following expression:

$$\mathbf{A} = [a_{ij}]_{m \times n}$$

Where  $a_{ij}$  is referred to as the  $(i,j)$  -element of the matrix  $\mathbf{A}$  . The subscript of  $m \times n$  can be dropped if the dimension of the matrix is clear from the context.

In deep learning neural network weights are stored as matrices, while feature inputs are stored as vectors. Formulating the problem in terms of linear algebra allows compact handling of these computations. By casting the problem in terms of tensors and utilising the machinery of linear algebra, rapid training times on modern GPU hardware can be obtained. Similarly, within ML, matrices serve as repositories for datasets, building kernel matrices for support vector machines, and implementing dimensionality reduction techniques such as principal component analysis. Within DS, matrices are indispensable for data preprocessing, transformation, and model assessment tasks.

**4. Tensors:** Tensors generalize the concept of vectors and matrices to higher dimensions. While scalars are zero-dimensional, vectors are one-dimensional, matrices are two-dimensional, and tensors can have any number of dimensions. Tensors are often used to represent multidimensional data, such as colour images, volumetric data, or time series data.

In machine learning, and deep learning in particular, a 3rd-order tensor can be used to describe the intensity values of multiple channels (red, green and blue) from a two-dimensional image.

### 3. Illustrate Matrices and Their Types:

#### Matrices and Their Types:

**Matrix:** A rectangular array of numbers, enclosed in a large pair of either parentheses or brackets, such as

$$\begin{pmatrix} 1 & 0 & -3 \\ -2 & 4 & 1 \end{pmatrix} \text{ or } \begin{bmatrix} 1 & 0 & -3 \\ -2 & 4 & 1 \end{bmatrix}$$

is called a **matrix**. The **size** or **dimensions** of a matrix are specified by stating the number of rows and the number of columns it contains. If the matrix consists of  $m$  rows and  $n$  columns, it is said to be an  **$m$  by  $n$** (written  $m \times n$ ) matrix. For example, the matrices above are 2 by 3, since they contain 2 rows and 3 columns:

$$\begin{array}{c}
 \text{row 1} \longrightarrow \left[ \begin{array}{ccc} 1 & 0 & -3 \end{array} \right] \\
 \text{row 2} \longrightarrow \left[ \begin{array}{ccc} -2 & 4 & 1 \end{array} \right] \\
 \begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ \text{columns 1, 2, 3} \end{array}
 \end{array}$$

Note that the rows are counted from top to bottom, and the columns are counted from left to right.

The numbers in the array are called the **entries** of the matrix, and the location of a particular entry is specified by giving first the row and then the column where it resides. The entry in row  $i$ , column  $j$  is called the  $(i, j)$  **entry**. For example, since the entry  $-2$  in the matrix above is in row 2, column 1, it is the  $(2, 1)$  entry. The  $(1, 2)$  entry is 0, the  $(2, 3)$  entry is 1, and so forth. In general, the  $(i, j)$  entry of a matrix  $A$  is written  $a_{ij}$ , and the statement

$$A = [a_{ij}]_{m \times n}$$

indicates that  $A$  is the  $m \times n$  matrix whose  $(i, j)$  entry is  $a_{ij}$ .

**Types of Matrices:** In linear algebra, there are many different types of matrices. All matrices are classified according to their elements, order, and a set of conditions. The plural version of a matrix is “matrices,” which is less usually used to refer to matrices.

1. **Row matrix:** A row matrix can be defined as the matrix which has only one row. In a row matrix, the number of columns is irrelevant; all that counts is that there is only one row. A single row makes up a row matrix.

Example:  $[2 \ 3 \ -1]$

2. **Column matrix:** A column matrix is similar to a row matrix, but with a few differences. The column matrix must have only one column to satisfy the criterion. It makes no difference how many rows there are in a column matrix; all that counts is that it has only one column.

Example:  $\begin{bmatrix} 6 \\ 6 \\ 7 \end{bmatrix}$

3. **Rectangular matrix:** A rectangle matrix has a different number of rows and columns than a square matrix, and its size is  $m \times n$ . Almost all matrices are rectangular in nature, but if the rows and columns are the same, the matrix is no longer rectangular.

Example:  $\begin{bmatrix} 1 & 2 & 5 \\ 4 & 38 \end{bmatrix}$

4. **Square matrix:** When the rows and columns are equal, the matrix becomes a square matrix rather than a rectangle matrix. The explanation for this is simple: a square has all equal sides, and when the number of rows and columns are equal, all sides of a matrix are equal, hence the name square

matrix. The primary diagonal is made up of parts of the form  $a_{ij}$ . The elements with  $i + j = n + 1$  create the secondary diagonal.

Example:  $\begin{bmatrix} 2 & 4 \\ -1 & -5 \end{bmatrix}$

5. **Upper triangular matrix:** Make a diagonal of the given matrix to comprehend the upper triangular matrix. You have an upper triangular matrix if all the entries below the diagonal are zero.

Example:  $\begin{bmatrix} 1 & 7 & -2 \\ 0 & -3 & 4 \\ 0 & 0 & 2 \end{bmatrix}$

6. **Lower triangular matrix:** The difference between a lower triangular matrix and an upper triangular matrix is the position of the zeros. All of the elements above the diagonal in the lower triangular matrix are zero. We'll call it a lower triangular matrix regardless of what's below the diagonal, as long as the components above the diagonal are zero; nevertheless, the diagonal elements will never be 0 in either the upper or lower triangular matrix.

Example:  $\begin{bmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ 3 & 56 & 1 \end{bmatrix}$

7. **Diagonal matrix:** All the elements above and below the diagonal in a diagonal matrix are zeros. It's similar to combining an upper triangular matrix with a lower triangular matrix.

Example:  $\begin{bmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 5 \end{bmatrix}$

8. **Scalar matrix:** A scalar matrix is similar to a diagonal matrix, but with one exception. The scalar matrix must satisfy two requirements. The first is that all elements should be zero, both above and below the diagonal. The second requirement is that the diagonal elements must be identical. As a result, a scalar matrix is a diagonal matrix with equal diagonal members.

Example:  $\begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix}$

9. **Identity matrix:** A diagonal matrix with all diagonal members is called an identity matrix.

Example:  $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

10. **Zero matrix:** All of the elements of a zero matrix are considered to be zero.

Example:  $\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

#### 4. Discuss about matrix operations with suitable examples

**Operations with Matrices:** As far as linear algebra is concerned, the two most important operations with vectors are vector addition [adding two (or more) vectors] and scalar multiplication (multiplying a vector by a scalar). Analogous operations are defined for matrices.

**1.Matrix addition.** If  $A$  and  $B$  are matrices of the same size, then they can be added. (This is similar to the restriction on adding vectors, namely, only vectors from the same space  $\mathbf{R}^n$  can be added; you cannot add a 2-vector to a 3-vector, for example.) If  $A = [a_{ij}]$  and  $B = [b_{ij}]$  are both  $m \times n$  matrices, then their sum,  $C = A + B$ , is also an  $m \times n$  matrix, and its entries are given by the formula

$$c_{ij} = a_{ij} + b_{ij}$$

Thus, to find the entries of  $A + B$ , simply add the corresponding entries of  $A$  and  $B$ .

**Example 1:** Consider the following matrices:

$$F = \begin{bmatrix} 2 & -1 \\ 3 & 0 \\ -5 & 2 \end{bmatrix}, \quad G = \begin{bmatrix} 4 & 4 & -3 \\ 0 & -1 & -2 \end{bmatrix}, \quad H = \begin{bmatrix} 1 & 6 \\ -1 & -2 \\ 0 & -3 \end{bmatrix}$$

Since only **matrices of the same size can be added**, only the sum  $F + H$  is defined ( $G$  cannot be added to either  $F$  or  $H$ ). The sum of  $F$  and  $H$  is

$$F + H = \begin{bmatrix} 2 & -1 \\ 3 & 0 \\ -5 & 2 \end{bmatrix} + \begin{bmatrix} 1 & 6 \\ -1 & -2 \\ 0 & -3 \end{bmatrix} = \begin{bmatrix} 2+1 & -1+6 \\ 3-1 & 0-2 \\ -5+0 & 2-3 \end{bmatrix} = \begin{bmatrix} 3 & 5 \\ 2 & -2 \\ -5 & -1 \end{bmatrix}$$

Since addition of real numbers is commutative, it follows that addition of matrices (when it is defined) is also commutative; that is, for any matrices  $A$  and  $B$  of the same size,  $A + B$  will always equal  $B + A$ .

**Example 2:** If any matrix  $A$  is added to the zero matrix of the same size, the result is clearly equal to  $A$ :

$$A + 0 = 0 + A = A$$

This is the matrix analog of the statement  $a + 0 = 0 + a = a$ , which expresses the fact that the number 0 is the additive identity in the set of real numbers.

**2. Scalar multiplication:** A matrix can be multiplied by a scalar as follows. If  $A = [a_{ij}]$  is a matrix and  $k$  is a scalar, then

$$kA = [ka_{ij}]$$

That is, the matrix  $kA$  is obtained by multiplying each entry of  $A$  by  $k$ .

**Example 3:** If

$$A = \begin{bmatrix} 1 & 0 & -3 \\ -2 & 4 & 1 \end{bmatrix}$$

then the scalar multiple  $2A$  is obtained by multiplying every entry of  $A$  by 2:

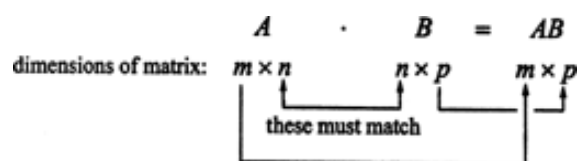
$$2A = \begin{bmatrix} 2 & 0 & -6 \\ -4 & 8 & 2 \end{bmatrix}$$

**3. Matrix multiplication:** By far the most important operation involving matrices is *matrix multiplication*, the process of multiplying one matrix by another. The first step in defining matrix multiplication is to recall the definition of the dot product of two vectors. Let  $\mathbf{r}$  and  $\mathbf{c}$  be two  $n$ -vectors. Writing  $\mathbf{r}$  as a  $1 \times n$  row matrix and  $\mathbf{c}$  as an  $n \times 1$  column matrix, the dot product of  $\mathbf{r}$  and  $\mathbf{c}$  is

$$\mathbf{r} \cdot \mathbf{c} = \begin{bmatrix} r_1 & r_2 & \cdots & r_n \end{bmatrix} \cdot \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix} = r_1c_1 + r_2c_2 + \cdots + r_nc_n$$

Note that in order for the dot product of  $\mathbf{r}$  and  $\mathbf{c}$  to be defined, both must contain the same number of entries. Also, the order in which these matrices are written in this product is important here: The row vector comes first, the column vector second.

In order to form the product  $AB$ , **the number of columns of  $A$  must match the number of rows of  $B$** ; if this condition does not hold, then the product  $AB$  is not defined. This criterion follows from the restriction stated above for multiplying a row matrix  $\mathbf{r}$  by a column matrix  $\mathbf{c}$ , namely that the number of entries in  $\mathbf{r}$  must match the number of entries in  $\mathbf{c}$ . If  $A$  is  $m \times n$  and  $B$  is  $n \times p$ , then the product  $AB$  is defined, and the size of the product matrix  $AB$  will be  $m \times p$ . The following diagram is helpful in determining if a matrix product is defined, and if so, the dimensions of the product:



Thinking of the  $m \times n$  matrix  $A$  as composed of the row vectors  $\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_m$  from  $\mathbf{R}^n$  and the  $n \times p$  matrix  $B$  as composed of the column vectors  $\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_p$  from  $\mathbf{R}^n$ ,

$$A = \begin{bmatrix} \text{row 1} \rightarrow \\ \text{row 2} \rightarrow \\ \vdots \\ \text{row } m \rightarrow \end{bmatrix} = \begin{bmatrix} \text{---} & \mathbf{r}_1 & \text{---} \\ \text{---} & \mathbf{r}_2 & \text{---} \\ & \vdots & \\ \text{---} & \mathbf{r}_m & \text{---} \end{bmatrix}$$

and

$$B = \begin{bmatrix} \downarrow & \downarrow & \vdots & \downarrow \\ \text{col 1} & \text{col 2} & & \text{col } p \end{bmatrix} = \begin{bmatrix} \downarrow & \downarrow & \vdots & \downarrow \\ \mathbf{c}_1 & \mathbf{c}_2 & & \mathbf{c}_p \end{bmatrix}$$

the rule for computing the entries of the matrix product  $AB$  is  $\mathbf{r}_i \cdot \mathbf{c}_j = (AB)_{ij}$ , that is,

The dot product of row  $i$  in  $A$  and column  $j$  in  $B$  gives the  $(i, j)$  entry of  $AB$

**Example 4:** Given the two matrices

$$A = \begin{bmatrix} 1 & 0 & -3 \\ -2 & 4 & 1 \end{bmatrix} \text{ and } B = \begin{bmatrix} 1 & 0 & 4 & 1 \\ -2 & 3 & -1 & 5 \\ 0 & -1 & 2 & 1 \end{bmatrix}$$

determine which matrix product,  $AB$  or  $BA$ , is defined and evaluate it.

Since  $A$  is  $2 \times 3$  and  $B$  is  $3 \times 4$ , the product  $AB$ , in that order, is defined, and the size of the product matrix  $AB$  will be  $2 \times 4$ . The product  $BA$  is *not* defined, since the first factor ( $B$ ) has 4 columns but the second factor ( $A$ ) has only 2 rows. The number of columns of the first matrix must match the number of rows of the second matrix in order for their product to be defined.

Taking the dot product of row 1 in  $A$  and column 1 in  $B$  gives the  $(1, 1)$  entry in  $AB$ . Since

$$\begin{bmatrix} 1 & 0 & -3 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ -2 \\ 0 \end{bmatrix} = (1)(1) + (0)(-2) + (-3)(0) = 1$$

the  $(1, 1)$  entry in  $AB$  is 1:

$$\begin{bmatrix} 1 & 0 & -3 \\ -2 & 4 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 4 & 1 \\ -2 & 3 & -1 & 5 \\ 0 & -1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 & & & \\ & & & \end{bmatrix}$$

The dot product of row 1 in  $A$  and column 2 in  $B$  gives the  $(1, 2)$  entry in  $AB$ ,

$$\begin{bmatrix} \boxed{1} & 0 & -3 \\ -2 & 4 & 1 \end{bmatrix} \begin{bmatrix} 1 & \boxed{0} & 4 & 1 \\ -2 & 3 & -1 & 5 \\ 0 & -1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 & \boxed{3} & & \end{bmatrix}$$

and the dot product of row 1 in  $A$  and column 3 in  $B$  gives the (1, 3) entry in  $AB$ :

$$\begin{bmatrix} \boxed{1} & 0 & -3 \\ -2 & 4 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & \boxed{4} & 1 \\ -2 & 3 & -1 & 5 \\ 0 & -1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 3 & \boxed{-2} & \end{bmatrix}$$

The first row of the product is completed by taking the dot product of row 1 in  $A$  and column 4 in  $B$ , which gives the (1, 4) entry in  $AB$ :

$$\begin{bmatrix} \boxed{1} & 0 & -3 \\ -2 & 4 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 4 & \boxed{1} \\ -2 & 3 & -1 & 5 \\ 0 & -1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 3 & -2 & \boxed{-2} \end{bmatrix}$$

Now for the second row of  $AB$ : The dot product of row 2 in  $A$  and column 1 in  $B$  gives the (2, 1) entry in  $AB$ ,

$$\begin{bmatrix} \boxed{1} & 0 & -3 \\ -2 & 4 & 1 \end{bmatrix} \begin{bmatrix} \boxed{1} & 0 & 4 & 1 \\ -2 & 3 & -1 & 5 \\ 0 & -1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} \boxed{1} & 3 & -2 & -2 \\ \boxed{-10} & & & \end{bmatrix}$$

and the dot product of row 2 in  $A$  and column 2 in  $B$  gives the (2, 2) entry in  $AB$ :

$$\begin{bmatrix} \boxed{1} & 0 & -3 \\ -2 & 4 & 1 \end{bmatrix} \begin{bmatrix} 1 & \boxed{0} & 4 & 1 \\ -2 & 3 & -1 & 5 \\ 0 & -1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 3 & -2 & -2 \\ -10 & \boxed{11} & & \end{bmatrix}$$

Finally, taking the dot product of row 2 in  $A$  with columns 3 and 4 in  $B$  gives (respectively) the (2, 3) and (2, 4) entries in  $AB$ :

$$\begin{bmatrix} \boxed{1} & 0 & -3 \\ -2 & 4 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & \boxed{4} & 1 \\ -2 & 3 & -1 & 5 \\ 0 & -1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 3 & -2 & -2 \\ -10 & 11 & \boxed{-10} & \end{bmatrix}$$

$$\begin{bmatrix} \boxed{1} & 0 & -3 \\ -2 & 4 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 4 & \boxed{1} \\ -2 & 3 & -1 & 5 \\ 0 & -1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 3 & -2 & -2 \\ -10 & 11 & -10 & \boxed{19} \end{bmatrix}$$

Therefore,

$$AB = \begin{bmatrix} \boxed{1} & 0 & -3 \\ -2 & 4 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 4 & 1 \\ -2 & 3 & -1 & 5 \\ 0 & -1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 3 & -2 & -2 \\ -10 & 11 & -10 & 19 \end{bmatrix}$$

**4. The inverse of a matrix:** Let  $a$  be a given real number. Since 1 is the multiplicative identity in the set of real numbers, if a number  $b$  exists such that

$$ab = ba = 1$$

then  $b$  is called the **reciprocal or multiplicative inverse** of  $a$  and denoted  $a^{-1}$  (or  $1/a$ ). The analog of this statement for square matrices reads as follows. Let  $A$  be a given  $n \times n$  matrix. Since  $I = I_n$  is the multiplicative identity in the set of  $n \times n$  matrices, if a matrix  $B$  exists such that

$$AB = BA = I$$

then  $B$  is called the (multiplicative) **inverse** of  $A$  and denoted  $A^{-1}$  (read “ $A$  inverse”).

#### **Method 1:**

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

The inverse of a matrix is found using the following formula:

$$A^{-1} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1}$$

$$A^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

#### **Method 2:**

The inverse matrix is also found using the following equation:

$$A^{-1} = \text{adj}(A) / \det(A),$$

where  $\text{adj}(A)$  refers to the adjoint of a matrix  $A$ ,  $\det(A)$  refers to the determinant of a matrix  $A$ .

**Example 5:** If

$$A = \begin{bmatrix} 2 & 3 \\ 5 & 8 \end{bmatrix}$$

then

$$A^{-1} = \begin{bmatrix} 8 & -3 \\ -5 & 2 \end{bmatrix}$$

since

$$AA^{-1} = \begin{bmatrix} 2 & 3 \\ 5 & 8 \end{bmatrix} \begin{bmatrix} 8 & -3 \\ -5 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I$$

and

$$A^{-1}A = \begin{bmatrix} 8 & -3 \\ -5 & 2 \end{bmatrix} \begin{bmatrix} 2 & 3 \\ 5 & 8 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I$$

Yet another distinction between the multiplication of scalars and the multiplication of matrices is provided by the **existence of inverses**. Although every nonzero real number has an inverse, *there exist nonzero matrices that have no inverse*.

**Example 6:** Show that the nonzero matrix

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$$

has no inverse.

If this matrix had an inverse, then

$$\begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} a & b \\ c & d \end{bmatrix} \text{ would equal } \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

for some values of  $a$ ,  $b$ ,  $c$ , and  $d$ . However, since the second row of  $A$  is a zero row, you can see that the second row of the product must also be a zero row:

$$\begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} * & * \\ 0 & \boxed{0} \end{bmatrix}$$

(When an asterisk, \*, appears as an entry in a matrix, it implies that the actual value of this entry is irrelevant to the present discussion.) Since the (2, 2) entry of the product cannot equal 1, the product cannot equal the identity matrix. Therefore, it is impossible to construct a matrix that can serve as the inverse for  $A$ . If a matrix has an inverse, it is said to be **invertible**.

**5. The transpose of a matrix:** One of the most basic operations that can be performed on a matrix is to form its transpose. Let  $A$  be a matrix; then the **transpose** of  $A$ , a matrix denoted by  $A^T$ , is obtained by writing the rows of  $A$  as columns. More precisely, row  $i$  of  $A$  is column  $i$  of  $A^T$  (which implies that column  $j$  of  $A$  is row  $j$  of  $A^T$ ). If  $A$  is  $m \times n$ , then  $A^T$  will be  $n \times m$ . Also, it follows immediately from the definition that  $(A^T)^T = A$ .

**Example 4:** The transpose of the  $2 \times 3$  matrix

$$A = \begin{bmatrix} 1 & 0 & -3 \\ -2 & 4 & 1 \end{bmatrix}$$

is the 3 x 2 matrix

$$A^T = \begin{bmatrix} 1 & -2 \\ 0 & 4 \\ -3 & 1 \end{bmatrix}$$

### 5. Elaborate Norms and its types with suitable diagrams?

**Norms:** A norm is a kind of function that measures the length of real vectors and matrices. The notion of length is extremely useful as it enables us to define distance - or similarity - between any two vectors (or matrices) living in the same space.

#### 1. Vector Norms

**1.1. The  $\ell_2$  Norm:** We begin with the most widely-used vector norm in machine learning, the  $\ell_2$  norm, defined for an N dimensional vector  $\mathbf{x}$  as

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{n=1}^N x_n^2}$$

Using the  $\ell_2$  norm we can measure the distance between any two points  $\mathbf{x}$  and  $\mathbf{y}$  via  $\|\mathbf{x}-\mathbf{y}\|_2$ , which is simply the length of the vector connecting  $\mathbf{x}$  and  $\mathbf{y}$ .

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \text{and} \quad \mathbf{y} = \begin{bmatrix} 9 \\ 8 \end{bmatrix}$$

For example, the distance between  $\mathbf{x}$  and  $\mathbf{y}$  is calculated as

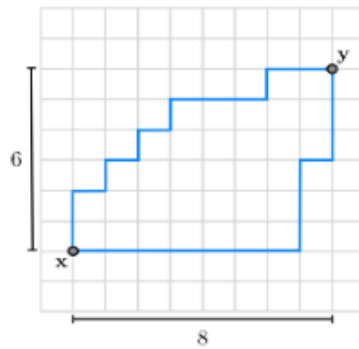
$\sqrt{(1-9)^2 + (2-8)^2} = 10$ , as shown pictorially in the figure below (recall Pythagorean theorem).

The  $\ell_2$  norm is not the only way to measure, or more precisely *define*, the length of a vector.

**1.2 The  $\ell_1$  norm:** The  $\ell_1$  norm of a vector  $\mathbf{x}$  is defined as the sum of the absolute values of its entries

$$\|\mathbf{x}\|_1 = \sum_{n=1}^N |x_n|$$

In terms of the  $\ell_1$  norm the distance between  $\mathbf{x}$  and  $\mathbf{y}$  is given by  $\|\mathbf{x}-\mathbf{y}\|_1$ , which provides a measurement of distance different from the  $\ell_2$  norm. As illustrated in the figure below the distance defined by the  $\ell_1$  norm is the length of a path consisting of perpendicular pieces (shown in blue). Because these paths are somewhat akin to how an automobile might travel from  $\mathbf{x}$  to  $\mathbf{y}$  if they were two locations in a gridded city, having to traverse perpendicular city blocks one after the other, the  $\ell_1$  norm is sometimes referred to as the taxicab norm, and the distance measured via the  $\ell_1$  norm, the Manhattan distance.



$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \text{and} \quad \mathbf{y} = \begin{bmatrix} 9 \\ 8 \end{bmatrix}$$

In the example above the distance between  $\ell_1$  norm is calculated as  $|1-9|+|2-8|=14$ .

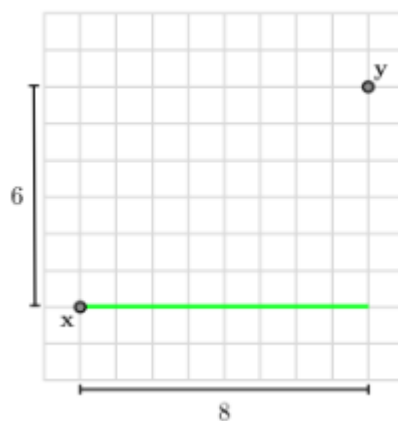
**1.3 The  $\ell_\infty$  norm:** The  $\ell_\infty$  norm of a vector  $\mathbf{x}$  is equal to its largest entry in terms of absolute value,

defined mathematically as  $\|\mathbf{x}\|_\infty = \max_n |x_n|$

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \text{and} \quad \mathbf{y} = \begin{bmatrix} 9 \\ 8 \end{bmatrix}$$

For example, the distance between in terms of the

$\ell_\infty$  norm is found as  $\max(|1-9|, |2-8|)=8$



**1.4 The  $\ell_p$  norm balls:** A norm ball is a set of all vectors  $\mathbf{x}$  with same norm value, that is, all  $\mathbf{x}$  such that  $\|\mathbf{x}\|=c$  for some constant  $c>0$ . When  $c=1$ , this set is called the unit norm ball, or simply the unit ball.

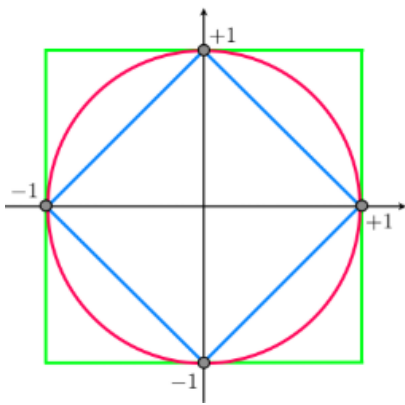
As you can see in the figure below, the  $\ell_1$  unit ball takes the form of a diamond (in blue), whose equation is given by

$$|x_1| + |x_2| = 1$$

The  $\ell_2$  unit ball is a circle (in red) defined by

$$x_1^2 + x_2^2 = 1$$

And finally as  $p \rightarrow \infty$  the unit ball approaches a square (in green) characterized by  $\max(|x_1|, |x_2|) = 1$



**Matrix Norm:** Matrix norms are a fundamental concept in linear algebra and are used to measure the size or magnitude of a matrix. They are essential in various applications, including numerical analysis, optimization, and machine learning. A matrix norm is a function that assigns a non-negative real number to a matrix, satisfying certain properties such as positivity, homogeneity, and the triangle inequality.

**2.1 The Frobenius norm:** The Frobenius norm is the intuitive extension of the  $\ell_2$  norm for vectors to matrices, defined similarly as the square root of the sum of the squares of all the elements in the matrix. Thus, the Frobenius norm of an  $N \times M$  dimensional matrix  $X$  is calculated as

$$\|\mathbf{X}\|_F = \sqrt{\sum_{n=1}^N \sum_{m=1}^M x_{n,m}^2}$$

$$\mathbf{X} = \begin{bmatrix} -1 & 2 \\ 0 & 5 \end{bmatrix}$$

For example, the Frobenius norm of the matrix

$$\sqrt{(-1)^2 + 2^2 + 0^2 + 5^2} = \sqrt{30}.$$

can be found as

The connection between the  $\ell_2$  norm and the Frobenius norm goes even deeper: collecting all singular values of  $\mathbf{X}$  in the vector  $\mathbf{s}$  (assuming  $N \leq M$ ).

$$\mathbf{s} = \begin{bmatrix} \sigma_1 \\ \sigma_2 \\ \vdots \\ \sigma_N \end{bmatrix}$$

The Frobenius norm of  $\mathbf{X}$  can be shown to be equal to the  $\ell_2$  norm of  $\mathbf{s}$ , i.e.,

$$\|\mathbf{X}\|_F = \|\mathbf{s}\|_2$$

## 2.1 The spectral and nuclear norms

The observation that the  $\ell_2$  norm of the vector of singular values of a matrix is identical to its Frobenius norm motivates the use of other  $\ell_p$  norms on the vector  $\mathbf{s}$ . In particular the  $\ell_1$  norm of  $\mathbf{s}$  defines the nuclear norm of  $\mathbf{X}$  denoted by  $\|\mathbf{X}\|_*$

$$\|\mathbf{X}\|_* = \|\mathbf{s}\|_1$$

and the  $\ell_\infty$  norm of  $\mathbf{s}$  defines the spectral norm of  $\mathbf{X}$ , denoted by  $\|\mathbf{X}\|_2$

$$\|\mathbf{X}\|_2 = \|\mathbf{s}\|_\infty$$

Because the singular values of real matrices are always non-negative, the spectral norm and the nuclear norm are simply the largest and the sum of the singular values, respectively.

**Applications and Importance:** Matrix norms have numerous applications in various fields, including:

- **Numerical analysis:** Matrix norms are used to estimate the error in numerical computations and to analyze the stability of algorithms.
- **Optimization:** Matrix norms are used to define regularization terms and to constraint the solution space in optimization problems.
- **Machine learning:** Matrix norms are used in regularization techniques, such as L1 and L2 regularization, to prevent overfitting in machine learning models.
- **Deep learning:** Matrix norms are used to regularize the weights of neural networks, preventing overfitting and improving generalization, used to improve the robustness of neural networks in adversarial attacks, In Principal Component Analysis (PCA) to reduce the dimensionality, and in Non-negative Matrix Factorization (NMF) to factorize non-negative matrices into lower-dimensional representations.

## 6. Explain Discuss about Matrix Decomposition?

**Matrix Decomposition** is a mathematical technique used to factorize a matrix into a product of simpler matrices, revealing its underlying structure and properties.

Factorization in deep learning is the mathematical process of decomposing a complex, high-dimensional dataset or weight matrix into several simpler, smaller matrices. It acts as a dimensionality reduction technique to compress models, extract hidden features (latent variables), and efficiently learn how different inputs interact

This decomposition enables us to simplify complex matrix operations, such as matrix inversion and determinant calculation, and provides valuable insights into the matrix's characteristics. The significance of Matrix Decomposition lies in its ability to:

- Simplify complex matrix operations
- Reveal the underlying structure and properties of a matrix
- Enable efficient computation of matrix inverses and determinants
- Facilitate data analysis and dimensionality reduction
- Enhance machine learning and signal processing techniques

### Types of Matrix Decomposition

There are several types of Matrix Decomposition, each with its unique characteristics and applications. The most commonly used decomposition techniques:

- a) LU Decomposition, Cholesky Decomposition,
- b) Eigen decomposition and
- c) Singular Value Decomposition (SVD).

Singular Value Decomposition (SVD) and Eigen Decomposition are two fundamental matrix factorization techniques used in linear algebra, data science, and machine learning and deep learning.

## 7. Discuss about Eigen Decomposition and Singular Value Decomposition?

**Eigen Decomposition:** It is the process of decomposing (factorizing) a matrix into **eigenvalues** and **eigenvectors**. Here, we break down our initial **square matrix** into the product of its eigenvalues and eigenvectors as defined in the following mathematical equation.

$$Ax = \lambda x$$

Here, **A** is an  $n \times n$  square matrix.  $\lambda$  is a scalar called the **eigenvalue** and **x** is a vector called the **eigenvector** with the corresponding  $\lambda$  value. The eigenvalues and eigenvectors come in pairs. Such a pair is known as an **eigenpair**. So, matrix **A** can have multiple such eigenpairs. The above equation shows the relationship between **A** and one of its eigenpairs.

**Steps in Eigen Decomposition:** To perform Eigen decomposition on a matrix, follow these steps:

**Step 1:** Find the Eigen values: Solve the characteristic equation:  $\det(A - \lambda I) = 0$

Here, A is the square matrix,  $\lambda$  is the eigen value, and I is the identity matrix of the same dimension as A.

**Step 2:** Find the Eigenvectors: For each eigen value  $\lambda$ , substitute it back into the equation:  $(A - \lambda I)v = 0$

This represents a system of linear equations where v is the eigenvector corresponding to the eigenvalue  $\lambda$ .

**Step 3:** Construct the Eigenvector Matrix V: Place all the eigenvectors as columns in the matrix V. If there are n distinct eigen values, V will be an  $n \times n$  matrix..

**Step 4:** Form the Diagonal Matrix  $\Lambda$ : Construct a diagonal matrix  $\Lambda$  by placing the eigenvalues on its diagonal:

**Step 5:** Calculate the Inverse of V: Find  $V^{-1}$ , the inverse of the eigenvector matrix invertible.

Find the eigenvectors corresponding to each eigenvalue

For,  $\lambda_1 = 5$ , solve:  $(A - 5I)v = 0$ :

$$\begin{bmatrix} -1 & 1 \\ 2 & -2 \end{bmatrix}$$

$$\Rightarrow \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = 0$$

$$\Rightarrow v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

For,  $\lambda_2 = 2$ , solve:  $(A - 2I)v = 0$ :

$$\begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix} = 0$$

$$\Rightarrow \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = 0$$

$$\Rightarrow v_2 = \begin{bmatrix} -\frac{1}{2} \\ 1 \end{bmatrix}$$

Form the matrix  $V$  of eigenvectors and  $\Lambda$  of eigenvalues

$$V = \begin{bmatrix} 1 & -\frac{1}{2} \\ 1 & 1 \end{bmatrix},$$

$$\Lambda = \begin{bmatrix} 5 & 0 \\ 0 & 2 \end{bmatrix}$$

Perform the Eigen decomposition

$$A = V\Lambda V^{-1}$$

Optionally, compute the inverse of  $V$

$$V^{-1} = \begin{bmatrix} \frac{2}{3} & \frac{1}{3} \\ -\frac{2}{3} & \frac{2}{3} \end{bmatrix}$$

**Singular Value Decomposition (SVD)**: It is a factorization method in linear algebra that decomposes a matrix into three other matrices, providing a way to represent data in terms of its singular values. SVD helps you split that table into three parts:

- **U**: This part tells you about the people (like their general preferences).
- **$\Sigma$** : This part shows how important each factor is (how much each rating matters).
- **$V^T$** : This part tells you about the products (how similar they are to each other)

Lets understand this with help of an example: Suppose you have a small table of people's ratings for two movies,

| Name   | Movie 1 Rating | Movie 2 Rating |
|--------|----------------|----------------|
| Amit   | 5              | 3              |
| Sanket | 4              | 2              |
| Harsh  | 2              | 5              |

- SVD breaks this table into three smaller parts: one that shows people's preferences, one that shows the importance of each movie, and one that shows how similar the movies are to each other
- Mathematically, the SVD of a matrix  $A$  (of size  $m \times n$ ) is represented as:  $A = U\Sigma V^T$

**Example:**

To perform Singular Value Decomposition (SVD) for the matrix  $A = \begin{bmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{bmatrix}$ , let's break it down step by step.

**Step 1: Compute  $AA^T$** 

First, we need to calculate the matrix  $AA^T$  (where  $A^T$  is the transpose of matrix  $A$ ):

$$A = \begin{bmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{bmatrix}$$

$$A^T = \begin{bmatrix} 3 & 2 \\ 2 & 3 \\ 2 & -2 \end{bmatrix}$$

Now, compute  $AA^T$ :

$$AA^T = \begin{bmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{bmatrix} \cdot \begin{bmatrix} 3 & 2 \\ 2 & 3 \\ 2 & -2 \end{bmatrix} = \begin{bmatrix} 17 & 8 \\ 8 & 17 \end{bmatrix}$$

**Step 2: Find the Eigenvalues of  $AA^T$** 

To find the eigenvalues of  $AA^T$ , we solve the characteristic equation:

$$\det(AA^T - \lambda I) = 0$$

$$\det \begin{bmatrix} 17 - \lambda & 8 \\ 8 & 17 - \lambda \end{bmatrix} = 0$$

$$(\lambda - 25)(\lambda - 9) = 0$$

Thus, the eigenvalues are  $\lambda_1 = 25$  and  $\lambda_2 = 9$ . These eigenvalues correspond to the singular values  $\sigma_1 = 5$  and  $\sigma_2 = 3$ , since the singular values are the square roots of the eigenvalues.

**Step 3: Find the Right Singular Vectors (Eigenvectors of  $A^T A$ )**

Next, we find the eigenvectors of  $A^T A$  for  $\lambda = 25$  and  $\lambda = 9$ .

**For  $\lambda = 25$ :**

Solve  $(A^T A - 25I)v = 0$ :

$$A^T A - 25I = \begin{bmatrix} -12 & 12 & 2 \\ 12 & -12 & -2 \\ 2 & -2 & -17 \end{bmatrix}$$

Row-reduce this matrix to:

$$\begin{bmatrix} 1 & -1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

The eigenvector corresponding to  $\lambda = 25$  is:

$$v_1 = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{bmatrix}$$

**For  $\lambda = 9$ :**

Solve  $(A^T A - 9I)v = 0$ :

The eigenvector corresponding to  $\lambda = 9$  is:

$$v_2 = \begin{bmatrix} \frac{1}{\sqrt{18}} \\ -\frac{1}{\sqrt{18}} \\ \frac{4}{\sqrt{18}} \end{bmatrix}$$

For the third eigenvector  $v_3$ :

Since  $v_3$  must be perpendicular to  $v_1$  and  $v_2$ , we solve the system  $v_1^T v_3 = 0$  and  $v_2^T v_3 = 0$ , leading to:

$$v_3 = \begin{bmatrix} \frac{2}{3} \\ -\frac{2}{3} \\ \frac{-1}{3} \end{bmatrix}$$

**Step 4: Compute the Left Singular Vectors (Matrix U)**

To compute the left singular vectors  $U$ , we use the formula  $u_i = \frac{1}{\sigma_i} A v_i$ . This results in:

$$U = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} \end{bmatrix}$$

**Step 5: Final SVD Equation**

Finally, the Singular Value Decomposition of matrix  $A$  is:

$$A = U\Sigma V^T$$

Where:

$$U = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} \end{bmatrix}$$

$$\Sigma = \begin{bmatrix} 5 & 0 & 0 \\ 0 & 3 & 0 \end{bmatrix}$$

$$V = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{18}} & \frac{-1}{\sqrt{18}} & \frac{4}{\sqrt{18}} \\ \frac{2}{3} & \frac{-2}{3} & \frac{1}{3} \end{bmatrix}$$

Thus, the SVD of matrix  $A$  is:

$$A = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} 5 & 0 & 0 \\ 0 & 3 & 0 \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{18}} & \frac{-1}{\sqrt{18}} & \frac{4}{\sqrt{18}} \\ \frac{2}{3} & \frac{-2}{3} & \frac{1}{3} \end{bmatrix}$$

**8. Discuss about Principal Component Analysis?**

**Principal Component Analysis:** Principal Component Analysis (PCA) is a dimensionality reduction technique commonly used in machine learning and data analysis to reduce the number of features while preserving as much information as possible. It achieves this by finding the principal components of the data, which are orthogonal vectors that capture the most significant variations in the data.

**Note:** In this example, all values are rounded to 2 decimal points.

**Steps for PCA:**

**1. Data Preparation:** Collect and preprocess your data.

**2. Covariance Matrix Computation:** Compute the covariance matrix of your centered data. The covariance matrix describes the relationships between the different features in your dataset.

**3. Eigenvalue and Eigenvector Computation:**

- Calculate the eigenvalues and eigenvectors of the covariance matrix. These represent the magnitude and direction of the principal components.
- The eigenvalues indicate the variance explained by each principal component, and the eigenvectors define the direction of these components.

**4. Sort Eigenvalues:** Sort the eigenvalues in descending order. The largest eigenvalues correspond to the most significant principal components.

**5. Select Principal Components:** Decide how many principal components to keep. You can select a specific number or choose based on the explained variance (e.g., keep components that explain 95% of the variance).

**6. Projection onto Principal Components:** Project your original data onto the selected principal components to obtain a new set of features (variables) that are linear combinations of the original features.

**Step — 1 Collect the data:** The following are the data points or features given. Let us start with 4 data points.

| Features | $E_1$ | $E_2$ | $E_3$ | $E_4$ |
|----------|-------|-------|-------|-------|
| $X_1$    | 2     | 3     | 4     | 5     |
| $X_2$    | 4     | 6     | 8     | 10    |

### Step — 2 Covariance Matrix Computation

$$S = \begin{bmatrix} \text{cov}(X_1, X_1) & \text{cov}(X_1, X_2) \\ \text{cov}(X_2, X_1) & \text{cov}(X_2, X_2) \end{bmatrix}$$

The final covariance matrix is the following

$$\text{COV matrix} = \begin{bmatrix} 1.83 & 3.33 \\ 3.33 & 6.67 \end{bmatrix}$$

**Step — 3 Eigen values and Eigen vector Computation:** Next we will compute the eigenvalues of this covariance matrix. The characteristic equation of the covariance matrix is the following.

To find the roots of the quadratic equation we can apply the following formula.

Quadratic Formula

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

By applying the formula, we will get two roots and these are the **eigen values**.

The next step is to calculate eigen vectors. To calculate this the initial step is to calculate the Characteristic Equation: The **characteristic equation** for finding eigenvalues ( $\lambda$ ) of a matrix C is given by:

$$\det(S - \lambda I) = 0$$

Where:

- S is the covariance matrix.
- $\lambda$  (lambda) represents the eigenvalues.
- I is the identity matrix of the same dimensions as C.

**Step — 4 Sort Eigen Values:** Whenever we want to calculate the principal components we have to consider the largest eigen value. Here it is 8.37. To calculate the unit eigen vector, we have to calculate the length of U1. To obtain unit eigenvectors (eigenvectors with a magnitude of 1), we normalize the eigenvectors. To normalize a vector, divide each component by its magnitude (Euclidean norm).

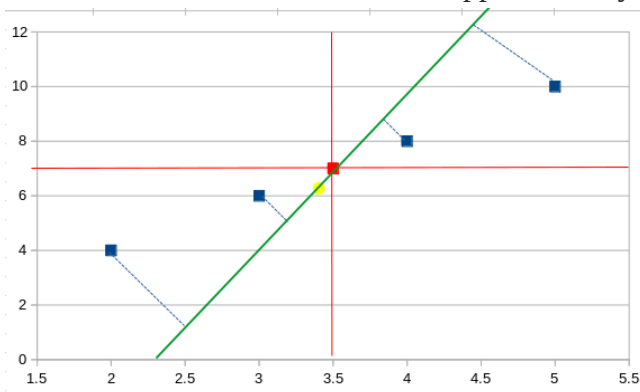
**Step — 5 Select First Principal Component:** The equation to calculate the principal component is as follows.

$$e_1^T \begin{bmatrix} X_{1k} - \bar{X}_1 \\ X_{2k} - \bar{X}_2 \end{bmatrix}$$

**Step — 6 Projection onto Principal Components:** Applying the equation on each data point. The final answer ,

|        |      |      |       |       |
|--------|------|------|-------|-------|
| $X_1$  | 2    | 3    | 4     | 5     |
| $X_2$  | 4    | 6    | 8     | 10    |
| $PC_1$ | 3.35 | 1.12 | -1.12 | -3.35 |

The final result can be visualized approximately like this.

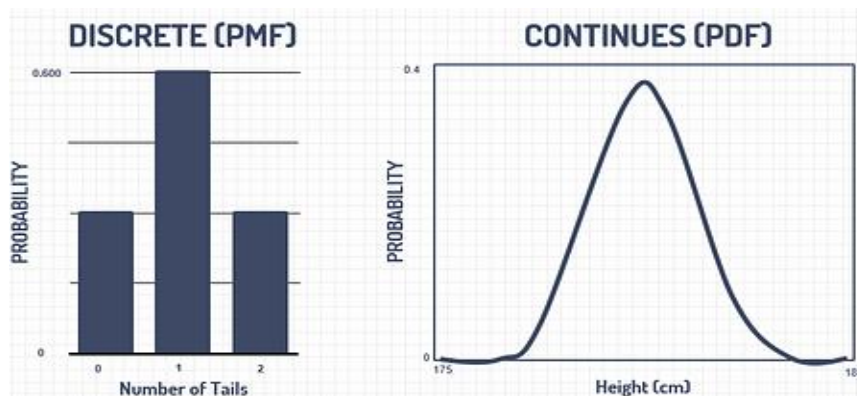


Blue points are the data points, the red point is the average point, the yellow circle is the eigenvector point  $(-0.45, -0.89)$  and the green line is the first principal component.

## 9. Discuss about Random variable?

**Random Variables:** A random variable, usually written  $X$ , is a **variable** whose possible values are numerical outcomes of a **random** phenomenon. There are two types of random variables:

1. Discrete Probability Mass Function (PMF)
2. Continuous Probability Density Function (PDF)



**1. Discrete:** The number of states are finite or countable infinite. *Example:* Let  $X$  represent the sum of two dice, the number of students in a class, test questions answered correctly, the number of children in a family, etc.

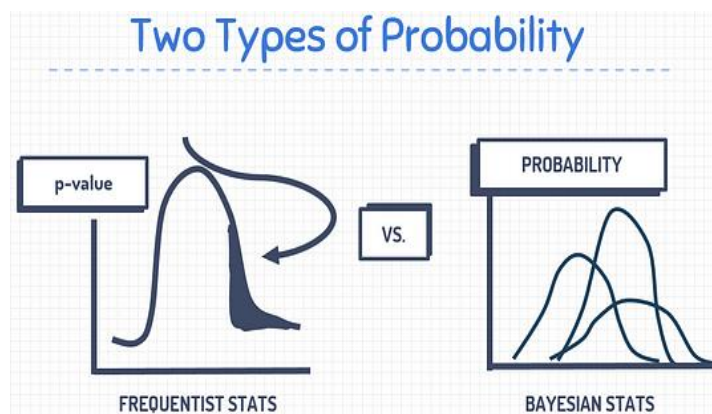
**2. Continuous:** It is associated with a real value. It is continuous between any two values of our variable or an infinite number of other valid values. Examples of continuous random variables are quantities such as pressure, height, mass, and distance.

### 10. Discuss about Probability distribution?

**Probability theory**, a branch of mathematics concerned with the analysis of random phenomena. It provides a mathematical framework for representing uncertainty, It Provides a means of quantifying uncertainty and axioms for deriving new uncertain statements. In the application of Artificial Intelligence, probability theory is used in two ways:

- ☒ It specifies how AI should handle reasoning.
- ☒ Use probability and statistics to analyze the behavior of proposed AI systems.

There are two types of probability:



**Frequentist Probability:** defines probability of some event in terms of the relative frequency with which the event tends to occur. It refers to the rates at which events occur.

**Bayesian Probability:** defines probability in more subjective terms — as a measure of the strength of your belief regarding the true situation. It refers to more qualitative levels of certainty.

**Probability Distribution:** Probability distributions describe how the probabilities of different outcomes are spread out over a specific sample space. When dealing with multiple variables, we look at their interactions through joint, marginal, and conditional probabilities to understand how one variable influences or relates to another.

**1. Probability Distributions:** A probability distribution is a mathematical function or table that shows the possible values of a variable and how often they occur.

- **Discrete distributions** (like a coin flip) use a Probability Mass Function (PMF).
- **Continuous distributions** (like height or time) use a Probability Density Function (PDF).

**2. Marginal Probability:** Marginal probability is the probability of a single event occurring, completely independent of any other events. To find it from a joint distribution, you "marginalize" by summing (or integrating) over all possible values of the other variables.

- **Formula:**  $P(X) = \sum_y P(X, y)$  (for discrete variables)
- **Example:** The chance it rains today, regardless of whether it is windy or not.

**3. Conditional Probability:** Conditional probability measures the likelihood of an event occurring *given* that another event has already happened. It updates our prediction based on new information.

- **Formula:**  $P(X|Y) = \frac{P(X \cap Y)}{P(Y)}$
- **Example:** The probability that the roads are slippery *given* that it is currently raining.

### How They Connect

The fundamental connection between all three is the product rule: **Joint Probability**  $P(X, Y)$  equals the **Marginal Probability** of one variable multiplied by the **Conditional Probability** of the other, written as  $P(X, Y) = P(X|Y)P(Y)$ .

### 11. Describe the Discrete Variables Probability Distribution

Or

### Describe the Bernoulli Distribution and Binomial Distribution?

**Discrete Variables Probability Distribution:** When dealing with Discrete variables – we need to describe the probability distribution with a **Probability Mass function (PMF)**. The **Probability Mass Function (PMF)** describes the probability distribution over a discrete random variable. In other terms, it is a function that returns the probability of a random variable being exactly equal to a specific value. The returned probability lies in the range  $[0, 1]$  and the sum of all probabilities for every state equals one. Three common discrete probability distributions: The Bernoulli, binomial and geometric distribution.

- **Bernoulli Distribution:** Named after the Swiss mathematician Jacob Bernoulli, the Bernoulli distribution is a discrete probability distribution of a single binary random variable, which either takes the value 1 or 0. Loosely speaking, we can think of the Bernoulli distribution as a model giving the set of possible outcomes for a single experiment that can be answered with a simple yes-no question. More formally the function can be stated as the following equation

$$f(k; p) = \begin{cases} q = 1 - p & \text{if } k=0 \\ p & \text{if } k=1 \end{cases}$$

$$f(k; p) = p^k (1 - p)^{1-k} \quad \text{for } k \in \{0, 1\}$$

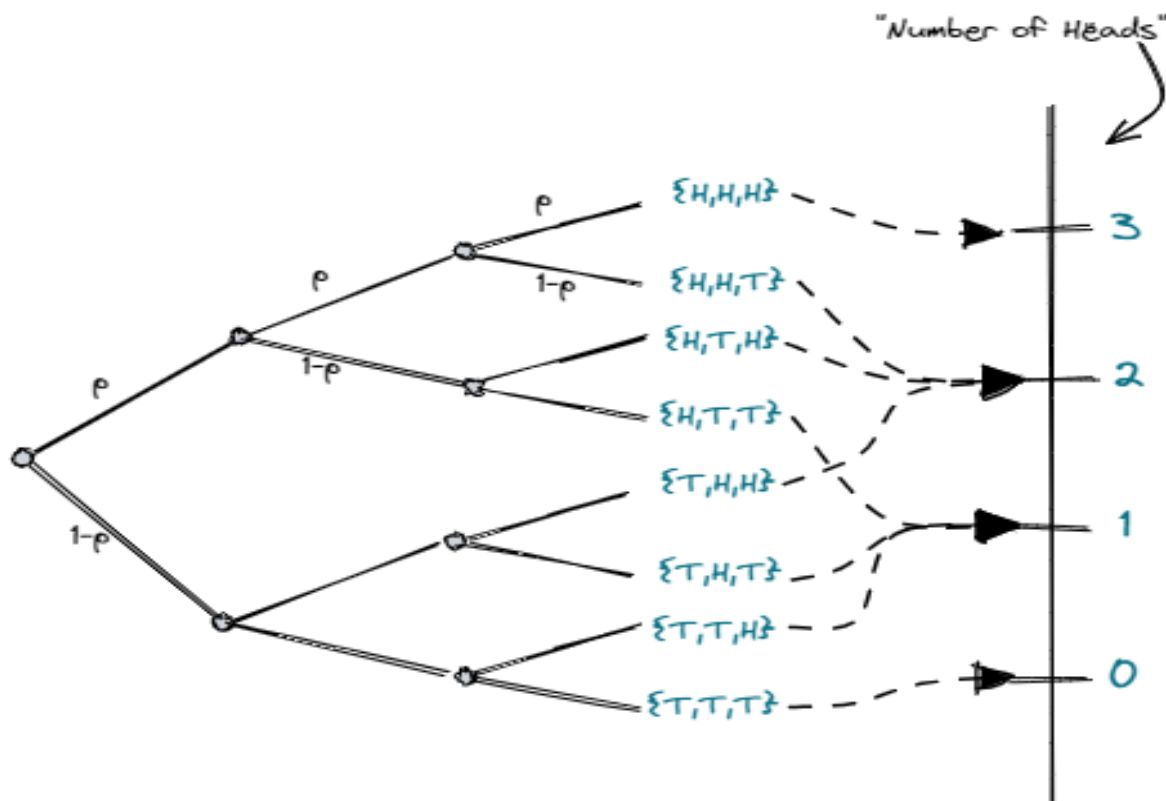
which basically evaluates to  $p$  if  $k=1$  or to  $(1-p)$  if  $k=0$ . Thus, the Bernoulli distribution is parameterized by just a **single parameter  $p$** . Suppose, we toss a fair coin once. The probability of obtaining heads is  $P(\text{Heads}) = 0.5$ .

- **Binomial Distribution:** The binomial distribution describes the discrete probability distribution of the number of successes in a sequence of  $n$  independent trials, each with a binary outcome. The success or failure is given by the probability  $p$  or  $(1-p)$  respectively. Thus, the binomial distribution is parameterized by the parameters  $n \in \mathbb{N}$ ,  $p \in [0, 1]$ . More formally the binomial distribution can be expressed with the following equation:

$$f(k; n, p) = \binom{n}{k} p^k (1-p)^{n-k}$$

The success of  $k$  is given by the probability  $p$  to the power of  $k$ , whereas the probability of failure is defined by  $(1-p)$  to the power of  $n$  minus  $k$ , which is basically the number of trials minus the one trial where we get  $k$ . Since the event of success  $k$  can \_\_\_ occur anywhere in  $n$  trials, we have " $n$  choose  $k$ " ways to distribute the success.

**Example:** Now, we are going to flip the fair coin three times, while being interested in the random variable describing the number of heads obtained.



If we want to compute the probability of the coin coming up as heads two times, we can simply use the equation from before and pluck in the values

$$P(2) = \binom{3}{2} p^2 (1-p)^{3-2}$$

$$P(2) = 3(0.5)^2 (0.5)^1$$

$$P(2) = 0.375$$

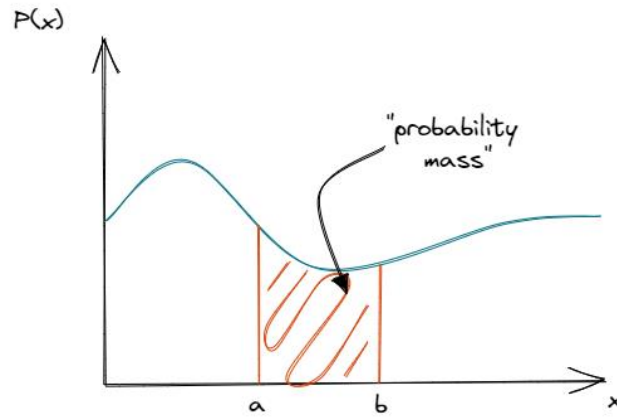
Which results in a probability  $P(2) = 0.375$ .

## 12. Describe Continuous Variables Probability Distribution?

OR

### Describe Gaussian Distribution or Normal Distribution?

**Continuous Variables Probability Distribution:** When dealing with continuous variables – we need to describe the probability distribution with a probability density function (PDF). The **PDF**, contrary to the PMF, does not give the probability of a random variable taking a specific state directly. Instead, it describes the probability of landing inside an infinitesimal region. In other terms, the PDF describes the probability of a random variable lying between a particular range of values. In order to find the actual probability mass, we need to integrate, which yields the **area under the density** function but above the x-axis. An example probability density function is shown below



The probability density function must be non-negative and its integral needs to be 1.

$$(1) \quad p(x) \geq 0$$

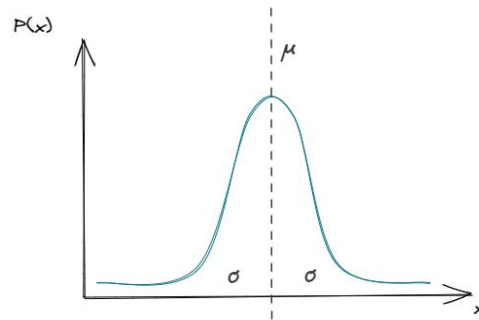
$$(2) \quad \int p(x) dx = 1$$

One of the most common continuous probability distributions is the Gaussian or normal distribution.

- **Gaussian Distribution or Normal Distribution:** The Gaussian distribution is often considered a sensible choice to represent a real-valued random variable, whose distribution is unknown. This is mainly due to the central limit theorem, which, loosely speaking, states that the average of many independent random variables with finite mean and variance is itself a random variable – which is normally distributed as the number of observations increases. This is especially useful since it allows us to model complicated systems as Gaussian distributed, even if the individual parts follow a more complicated structure or distribution. Another reason it is a common choice for modeling a distribution over a continuous variable is the fact that it inserts the least amount of prior knowledge. More formally, the Gaussian distribution can be stated as

$$\mathcal{N}(x; \mu, \sigma^2) = \sqrt{\frac{1}{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(x - \mu)^2\right)$$

where the parameter  $\mu$  is the mean and  $\sigma^2$  describes the variance. In simple terms, the mean will be responsible for defining the central peak of the bell-shaped distribution, whereas the variance or the standard deviation defines its width. We can visualize the normal distribution as the following:



### 13. Discuss about Probability of Multiple Random Variables

**OR**

### Discuss about Marginal Probability and Conditional Probability?

In machine learning and in deep learning, we are likely to work with many random variables. For example, given a table of data, such as in excel, each row represents a separate observation or event, and each column represents a separate random variable. Variables may be either discrete, meaning that they take on a finite set of values, or continuous, meaning they take on a real or numerical value.

When we have **two or more uncertain things happening** (which we call random variables), figuring out their probabilities together gets complicated. That's because these uncertain things can **affect each other** in various ways, and how they interact directly changes the likelihood of different outcomes.

This can be simplified by reducing the discussion to just two random variables ( $X, Y$ ), although the principles generalize to multiple variables. And further, to discuss the probability of just two events, one for each variable ( $X=A, Y=B$ ), although we could just as easily be discussing groups of events for each variable. Therefore, we will introduce the probability of multiple random variables as the probability of event  $A$  and event  $B$ , which in shorthand is  $X=A$  and  $Y=B$ . We assume that the two variables are related or dependent in some way. As such, there are three main types of probability we might want to consider; they are:

- **Joint Probability:** Probability of events  $A$  and  $B$ .
- **Marginal Probability:** Probability of event  $X=A$  given variable  $Y$ .
- **Conditional Probability:** Probability of event  $A$  given event  $B$ .

These types of probability form the basis of much of predictive modeling with problems such as classification and regression. For example:

- The probability of a row of data is the joint probability across each input variable.
- The probability of a specific value of one input variable is the marginal probability across the values of the other input variables.
- The predictive model itself is an estimate of the conditional probability of an output given an input example.

**Marginal Probability** :Marginal probability or marginal probability distribution is a subset of a collection of random variables. If  $X$  and  $Y$  are discrete random variables and  $f(x,y)$  is the value of their joint probability distribution at  $(x,y)$ , the functions given by:

$$g(x) = \sum_y f(x,y) \text{ and}$$

$$h(y) = \sum_x f(x,y), \text{ are the marginal distributions of } X \text{ and } Y, \text{ respectively.}$$

A frequency table can be used to find the margins of probability.

| i/j      | 1    | 2    | 3    | 4    | 5    | 6    | $p_X(i)$ |
|----------|------|------|------|------|------|------|----------|
| 1        | 1/36 | 1/36 | 1/36 | 1/36 | 1/36 | 1/36 | 1/6      |
| 2        | 1/36 | 1/36 | 1/36 | 1/36 | 1/36 | 1/36 | 1/6      |
| 3        | 1/36 | 1/36 | 1/36 | 1/36 | 1/36 | 1/36 | 1/6      |
| 4        | 1/36 | 1/36 | 1/36 | 1/36 | 1/36 | 1/36 | 1/6      |
| 5        | 1/36 | 1/36 | 1/36 | 1/36 | 1/36 | 1/36 | 1/6      |
| 6        | 1/36 | 1/36 | 1/36 | 1/36 | 1/36 | 1/36 | 1/6      |
| $p_Y(j)$ | 1/6  | 1/6  | 1/6  | 1/6  | 1/6  | 1/6  |          |

A marginal distribution gets its name because it appears in the margins of a probability distribution table.

Marginal distributions follow a couple of rules:

- ⊗ The distribution must be from bivariate data. Bivariate is just another way of saying “two variables,” like X and Y. In the table above, the random variables i and j are coming from the roll of two dice.
- ⊗ A marginal distribution is where you are only interested in one of the random variables . In other words, either X or Y. In the above probability table, the sum probabilities of one variable are listed in the bottom row and the other sum probabilities are listed in the right column. So this table has two marginal distributions.

**Conditional Probability:** Conditional probability is one of the most important concepts in probability theory. It's the probability of one event occurring with some relationship to one or more other events. For example:

- ⊗ Event A is that it is snowing outside, and it has a 0.3 (30%) chance of snow today.
- ⊗ Event B is that you will need to go outside, and that has a probability of 0.5 (50%).

A conditional probability would look at these two events in relationship with one another, such as the probability that it is both snowing and you will need to go outside. The formula for conditional probability is:

$$P(B|A) = P(A \text{ and } B) / P(A), \text{ We can also rewrite as:}$$

$$P(B|A) = P(A \cap B) / P(A).$$

The chain rule of conditional probabilities is really important . It permits the calculation of any member of the joint distribution of a set of random variables using only conditional probabilities.

#### 14. Elaborate the following

a. Expectation

b. Variance

c. Covariance

In information theory, expectation, variance, and covariance are fundamental concepts used to analyze and understand the behavior of random variables, particularly within the context of information content and its fluctuations.

**Expectation:** Expectation is the average value of some random variable X. It is also called the expected value. The mean, expected value, or expectation of a random variable X is written as  $E(X)$  or  $\mu_X$ . If we observe N random values of X, then the mean of the N values will be approximately equal to  $E(X)$  for large N. The expectation is defined differently for continuous and discrete random variables.

*Definition:* Let  $X$  be a continuous random variable with p.d.f.  $f_X(x)$ . The expected value of  $X$  is

$$\mathbb{E}(X) = \int_{-\infty}^{\infty} x f_X(x) dx.$$

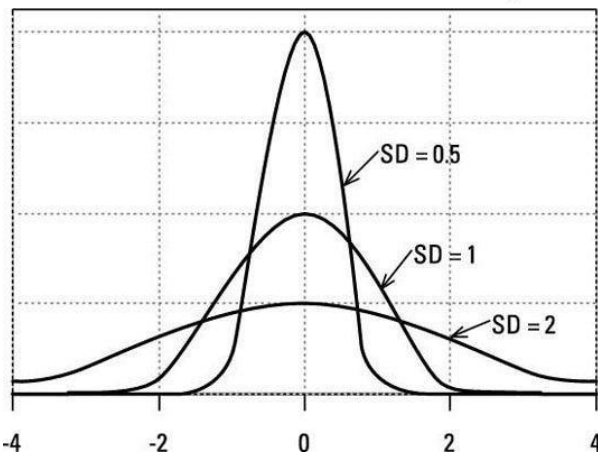
*Definition:* Let  $X$  be a discrete random variable with probability function  $f_X(x)$ . The expected value of  $X$  is

$$\mathbb{E}(X) = \sum_x x f_X(x) = \sum_x x \mathbb{P}(X = x).$$

**Variance:** Variance of a random variable  $X$  is a measure of how spread out it is. Are the values of  $X$  clustered tightly around their mean, or can we commonly observe values of  $X$  a long way from the mean value? The variance measures how far the values of  $X$  are from their mean, on average.

*Definition:* Let  $X$  be any random variable. The variance of  $X$  is

$$\text{Var}(X) = \mathbb{E}\left((X - \mu_X)^2\right) = \mathbb{E}(X^2) - \left(\mathbb{E}(X)\right)^2.$$



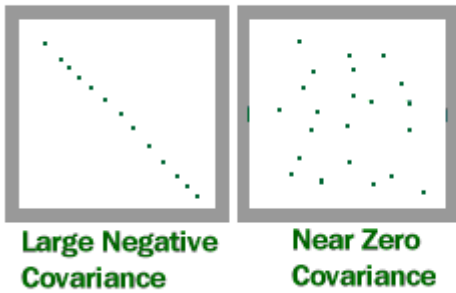
Lower the variance, more peakier the distribution. The square root of variance is called **standard deviation**.

**Covariance:** Covariance is a measure of the association or dependence between two random variables  $X$  and  $Y$ . Covariance can be either positive or negative. (Variance is always positive.)

*Definition:* Let  $X$  and  $Y$  be any random variables. The covariance between  $X$  and  $Y$  is given by

$$\text{cov}(X, Y) = \mathbb{E}\left\{(X - \mu_X)(Y - \mu_Y)\right\} = \mathbb{E}(XY) - \mathbb{E}(X)\mathbb{E}(Y),$$

where  $\mu_X = \mathbb{E}(X)$ ,  $\mu_Y = \mathbb{E}(Y)$ .

**COVARIANCE**

$$P(E_i | E) = \frac{P(E|E_i) \cdot P(E_i)}{P(E)}$$

**15. Discusses about Bayes' Theorem**

Bayes' Theorem is a fundamental concept in probability theory and statistics that plays a crucial role in decision-making under uncertainty. It allows us to update the probability of an event based on new evidence, helping in fields ranging from medical diagnosis to finance and artificial intelligence.

Bayes' Theorem describes how to compute the probability of a hypothesis given some evidence. It helps in understanding the relationship between prior beliefs (initial assumptions) and new evidence to compute a more accurate probability (posterior probability).

The formula for Bayes' Theorem is:

Where:

- $P(E_i | E)$  is the posterior probability (the probability of event  $E_i$  given that  $E$  is true).
- $P(E | E_i)$  is the likelihood (the probability of event  $E$  given that  $E_i$  is true).
- $P(E_i)$  is the prior probability (the initial probability of event  $E_i$ ).
- $P(E)$  is the marginal likelihood (the total probability of event  $E$ ).

## Bayes' Theorem: State and Prove

### Statement

Bayes' Theorem relates the conditional probability of two events and is stated as follows:

$$P(A | B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

### Proof:

From the definition of conditional probability:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

$$P(B | A) = \frac{P(A \cap B)}{P(A)}$$

Rearranging the second equation gives:

$$P(A \cap B) = P(B | A) \cdot P(A)$$

Substituting this into the first equation, we get:

$$P(A | B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

This is the **Bayes' Theorem** formula.

**Example 1:** There are three identical boxes, labeled I, II, and III, each containing two coins. Box I holds two gold coins, Box II holds two silver coins, and Box III contains one gold and one silver coin. A person randomly selects a box and draws a coin, which turns out to be gold. What is the probability that the other coin in the selected box is also gold?

**Solution:**

Let  $E_1$ ,  $E_2$  and  $E_3$  be the events that boxes I, II and III are chosen, respectively.

Then  $P(E_1) = P(E_2) = P(E_3) = \frac{1}{3}$

Also, Let A represent the event that "a gold coin is drawn."

Then

$P(A|E_1) = P(\text{drawing a gold coin from box I}) = \frac{2}{2} = 1$

$P(A|E_2) = P(\text{drawing a gold coin from box II}) = 0$

$P(A|E_3) = P(\text{drawing a gold coin from box III}) = \frac{1}{2}$

Now, the probability of the other coin in the box is also gold

= the probability of drawing a gold coin from box I.

=  $P(E_1 | A)$

By Bayes theorem, we know that

$$P(E_1 | A) = \frac{P(E_1)P(A|E_1)}{P(E_1)P(A|E_1) + P(E_2)P(A|E_2) + P(E_3)P(A|E_3)}$$

$$= \frac{\left(\frac{1}{3} \times 1\right)}{\left(\frac{1}{3} \times 1\right) + \left(\frac{1}{3} \times 0\right) + \left(\frac{1}{3} \times \frac{1}{2}\right)} = \frac{2}{3}$$

Hence, the required probability is  $\frac{2}{3}$ .

## 16. Discuss about Numerical Computation Overflow and Underflow?

### Numerical Computation: Overflow and Underflow

In Numerical computations, **overflow and underflow** are two common issues that can lead to unexpected results, from errors in your program to subtle inaccuracies that can snowball into significant problems. These phenomena are especially important when working with algorithms in areas like machine learning, deep learning, and scientific computing. Overflow and underflow are both errors resulting from a shortage of space. On the most basic level, they manifest in data types like integers and floating points.

Unlike the physical world, the number stored in a computer exists in a discrete number of digits. When we make a calculation that results in an extra digit, we cannot simply append that to our result, so we get an overflow or underflow error. Overflow errors come up when working with integers and floating points, and underflow errors are generally just associated with floating points.

**Overflow:** When we are trying to calculate a number that's just too big for your computer to handle. For instance, when calculating the value of  $e^{1000}$  (the exponential of a very large number), the result can exceed the maximum value that the computer's floating-point number system can represent. This results in overflow — a situation where numbers become so large that they “spill over” and are assigned an unrealistic value like infinity ( $\infty$ ). This can lead to all sorts of strange behavior in your program, as any further operations involving these values may yield unpredictable results.

Overflow indicates that we have done a calculation that resulted in a number larger than the largest number we can represent. Let's look at an example involving unsigned integers.

**Example:** Let's assume we have an integer stored in 1 byte. The greatest number we can store in one byte is 255, so let's take that. This is 11111111. Now, suppose we add 2 to it to get 00000010. The result is 257, which is 100000001. The result has 9 bits, whereas the integers we are working with consist of only 8. Then the computer will discard the most-significant bit (MSB) and keep the rest. This is essentially equal to  $r \% 2^n$ . Here,  $r$  is the result,  $n$  is the number of bits available, and  $\%$  is the modulo operator.

When very large numbers (too large for your computer's memory) are computed, they are approximated to infinity or minus infinity. This can be considered as a more serious approximation error than underflow, since it is a less good approximation. This may cause your algorithm to crash as well, even if it worked fine in theory. When you use the ML libraries for your models, they are often coded in a robust way to try to avoid underflow and overflow.

**Underflow:** On the opposite end of the spectrum, we have **underflow**. This happens when numbers become so small that they can't be represented accurately by the computer. Underflow is a bit trickier to understand because it has to do with precision in floating points. Again, due to the discrete nature of storage in computers, we cannot store an arbitrarily small number either. The floating-point convention comes up with techniques to represent fractional numbers. When we use these in calculations that result in a smaller number than our least value, we again exceed our designated space.

Underflow occurs most often when working with decimal numbers. Working with such small numbers is actually often the case as probabilities (especially joint probabilities) computed over and over again can be extremely small. Although underflow is a relatively harmless approximation error compared to overflow (since using zero as the approximation of very small numbers around zero is actually quite logical), it may lead to the unfortunate situation that your algorithm should work fine in theory, but you can't make it work in practice.

**Example:** Suppose we are given designated boxes to write decimal numbers in. We have one box on the left of the decimal point and three boxes on the right. So, we can easily represent 0.004.

Now, we want to perform a calculation,  $0.004 \times 0.004$ . The answer to this is 0.000016, but we simply do not have this many places available to us. So we discard the least-significant bits and store 0.000, which is quite obviously an erroneous answer.

### 17. Discuss about Preventing Overflow and Underflow?

**Discuss about Preventing Overflow and Underflow:** The key to solving these problems is **numerical stability**. Here are a few practical techniques you can use to handle overflow and underflow in your computations:

**1. Softmax Stability Trick:** A classic example where both overflow and underflow can cause issues is the **softmax function** used in machine learning, particularly in classification tasks. To make it numerically stable, you can **subtract the maximum value** of the input vector from all the elements of the vector before applying the exponential function. This shifts all values to ensure that no exponent gets too large (for overflow), and no exponent gets too small (for underflow).

**2. Use Logarithmic Transformations:** Another trick to prevent overflow and underflow is to use **logarithmic transformations** in calculations that involve exponentiation. For instance, instead of directly computing  $e^z$  you can compute  $\log(e^z)$ , which simplifies the calculations and avoids excessively large or small numbers. This is especially useful in optimization problems or likelihood functions where large exponents are common.

**3. Work with Higher Precision:** When you're dealing with numbers that can potentially overflow or underflow, you can use **higher precision** data types. For example, many programming languages offer **double precision** floating-point numbers, which provide more bits for representation, reducing the risk of overflow and underflow. In certain applications, even higher precision formats like **arbitrary-precision arithmetic** (e.g., `mpmath` in Python) can be useful, though these come with trade-offs in terms of computational speed.

**4. Scaling the Inputs:** Sometimes, scaling your inputs can help prevent overflow and underflow. If your data spans a large range (for example, very large positive and negative values), you can **normalize** or **standardize** the inputs before applying functions like softmax or activation functions in neural networks. This will ensure that the values stay within a manageable range.

### 18. Elaborate Optimization in Linear algebra?

**Optimization in Linear algebra:** Linear algebra is a branch of mathematics that deals with the study of linear equations, vector spaces, and linear transformations. It provides a powerful framework for representing and manipulating systems of linear equations, and is widely used in many areas of science and engineering.

**Optimization**, on the other hand, is the process of finding the best solution to a problem, subject to certain constraints. In the context of linear algebra, optimization involves finding the optimal values of variables that minimize or maximize a linear or nonlinear objective function, subject to linear or nonlinear constraints.

### Importance of Linear Algebra in Optimization

Linear algebra plays a crucial role in optimization because many optimization problems can be formulated and solved using linear algebraic techniques. In particular, linear algebra provides a powerful framework for representing and manipulating the constraints and objective functions of optimization problems. Some of the key reasons why linear algebra is important in optimization include:

- **Representation of constraints:** Linear algebra provides a convenient way to represent the constraints of an optimization problem, such as linear equality and inequality constraints.
- **Efficient computation:** Linear algebraic techniques can be used to efficiently compute the optimal solution to an optimization problem, especially for large-scale problems.
- **Analytical insights:** Linear algebra provides a framework for analyzing the properties of optimization problems, such as the existence and uniqueness of optimal solutions.

### 19. Discuss about Optimization Techniques in Linear Algebra?

**Optimization Techniques in Linear Algebra:** The key optimization techniques in linear algebra are

1. Gradient descent
2. Linear programming
3. Quadratic programming.

- **Gradient Descent and its Variants:** Gradient descent is a popular optimization algorithm that is widely used in machine learning and other fields. It is an iterative algorithm that uses the gradient of the objective function to update the parameters of the model. The basic idea behind gradient descent is to minimize the objective function by iteratively updating the parameters in the direction of the negative gradient. The update rule for gradient descent is given by:

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \alpha \nabla f(\mathbf{w}_k)$$

where  $\mathbf{w}_k$  is the current estimate of the parameters,  $\alpha$  is the learning rate, and  $\nabla f(\mathbf{w}_k)$  is the gradient of the objective function at  $\mathbf{w}_k$ .

**Some of the key variants of gradient descent include:**

- **Stochastic gradient descent:** This is a variant of gradient descent that uses a single example from the training dataset to compute the gradient, rather than the entire dataset.
- **Mini-batch gradient descent:** This is a variant of gradient descent that uses a small batch of examples from the training dataset to compute the gradient.
- **Momentum gradient descent:** This is a variant of gradient descent that adds a momentum term to the update rule to help escape local minima.

**Linear Programming and its Applications:** Linear programming is a type of optimization problem that involves minimizing or maximizing a linear objective function, subject to linear constraints. The general form of a linear programming problem is given by:

$$\min_{\mathbf{x}} \quad \mathbf{c}^T \mathbf{x} \text{ s.t.} \quad \mathbf{Ax} \leq \mathbf{b} \quad \mathbf{x} \geq 0$$

where  $\mathbf{x}$  is the vector of decision variables,  $\mathbf{c}$  is the vector of coefficients of the objective function,  $\mathbf{A}$  is the matrix of coefficients of the constraints, and  $\mathbf{b}$  is the vector of right-hand side values of the constraints. Linear programming has numerous applications in various fields, including finance, logistics, and energy management. Some of the key applications of linear programming include:

- **Portfolio optimization:** Linear programming can be used to optimize the allocation of assets in a portfolio to maximize returns or minimize risk.
- **Resource allocation:** Linear programming can be used to allocate resources, such as labor and materials, to different projects or tasks to maximize efficiency or minimize cost.
- **Supply chain management:** Linear programming can be used to optimize the flow of goods and services through a supply chain to minimize cost or maximize efficiency.
- **Quadratic Programming and its Applications:** Quadratic programming is a type of optimization problem that involves minimizing or maximizing a quadratic objective function, subject to linear constraints. The general form of a quadratic programming problem is given by:

$$\min_{\mathbf{x}} \quad \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{c}^T \mathbf{x} \text{ s.t.} \quad \mathbf{Ax} \leq \mathbf{b} \quad \mathbf{x} \geq 0$$

where  $\mathbf{Q}$  is a symmetric matrix that represents the quadratic term in the objective function. Quadratic programming has numerous applications in various fields, including finance, control systems, and machine learning. Some of the key applications of quadratic programming include:

- **Markowitz mean-variance optimization:** Quadratic programming can be used to optimize the allocation of assets in a portfolio to maximize returns or minimize risk.
- **Model predictive control:** Quadratic programming can be used to optimize the control inputs to a system to minimize a quadratic cost function.
- **Support vector machines:** Quadratic programming can be used to train support vector machines, which are a type of machine learning model.

## 20. Elaborate Constrained Optimization?

**Constrained optimization** (also known as '*Constraint Optimization*') is the process of finding the maximum or minimum value of a function subject to constraints. These constraints can take various forms, such as linear, nonlinear, equality or inequality constraints.

**Relationship between Constrained Optimization and Linear Algebra:** Linear algebra is related to constrained optimization because many optimization problems involve linear algebraic operations and techniques. Linear algebra provides tools for representing, manipulating, and analyzing the objective function and constraints in optimization problems.

**For example,** *linear programming* is a type of constrained optimization problem where the objective function and constraints are all linear functions of the variables. Linear algebra provides tools

for representing these linear functions in matrix form, which enables efficient computation and analysis. Linear algebra also provides techniques for solving systems of linear equations and inequalities, which are often used in optimization problems.

Moreover, linear algebra is used in the development and analysis of algorithms for solving optimization problems. For instance, many optimization algorithms rely on linear algebraic operations such as matrix-vector multiplication, matrix factorization, and eigenvalue calculations. These techniques can help to solve optimization problems more quickly and accurately.

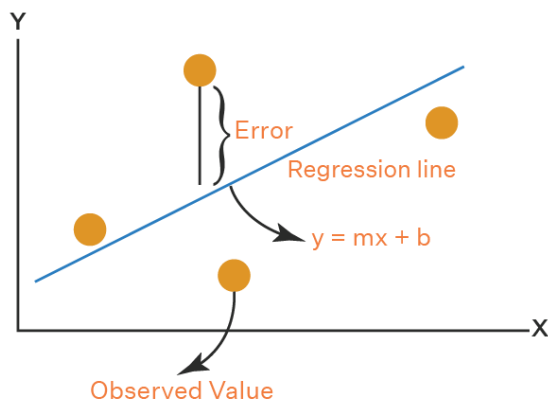
**Tools provided by Linear Algebra to solve Constrained Optimization problems:** Linear algebra provides a range of tools that are used to solve constrained optimization problems. Here are some of the most common tools:

1. **Matrices:** Matrices are used to represent systems of linear equations and inequalities, as well as linear transformations. They are also used to express the objective function and the constraints in linear programming problems.
2. **Vector spaces:** Vector spaces are used to define the space of feasible solutions in optimization problems. They provide a framework for expressing constraints on the variables, such as bounds on their values or linear equality constraints.
3. **Linear transformations:** Linear transformations are used to express the relationships between the variables in an optimization problem. They can be used to transform the original problem into a simpler form, such as a canonical form, which is easier to solve.
4. **Eigenvalues and eigenvectors:** Eigenvalues and eigenvectors are used to analyze the behavior of linear transformations. They can be used to identify the direction and rate of change of the objective function and constraints in an optimization problem.
5. **Matrix factorization:** Matrix factorization techniques, such as LU decomposition and Cholesky decomposition, are used to solve systems of linear equations and inequalities efficiently. They can also be used to identify the structure of the problem and to develop efficient algorithms for solving it.
6. **Convexity:** Linear algebra is used to study the properties of convex functions and sets, which are important in optimization problems. Convexity is a property that guarantees that a local minimum is also a global minimum, which makes it easier to solve optimization problems.

## 21. Discuss about Linear Least Square?

**Linear Least Square:** The least-squares method is a statistical method used to find the line of best fit of the form of an equation such as  $y = mx + b$  to the given data. The curve of the equation is called the regression line. Our main objective in this method is to reduce the **sum** of the squares of errors as much as possible. This is the reason this method is called the least-squares method. This method is often used in data fitting where the best fit result is assumed to reduce the sum of squared errors that is considered to be the difference between the observed values and corresponding fitted value. The sum of squared errors helps in finding the variation in observed data. For example, we have 4 data points and using this method we arrive at the following graph.

## Least Square Method



The two basic categories of least-square problems are ordinary or linear least squares and nonlinear least squares.

**Limitations for Least Square Method:** Even though the least-squares method is considered the best method to find the line of best fit, it has a few limitations. They are:

- This method exhibits only the relationship between the two variables. All other causes and effects are not taken into consideration.
- This method is unreliable when data is not evenly distributed.
- This method is very sensitive to outliers. In fact, this can skew the results

**Least Square Method Formula:** Least-square method is the curve that best fits a set of observations with a minimum sum of squared residuals or errors. Let us assume that the given points of data are  $(x_1, y_1)$ ,  $(x_2, y_2)$ ,  $(x_3, y_3)$ , ...,  $(x_n, y_n)$  in which all  $x$ 's are independent variables, while all  $y$ 's are dependent ones. This method is used to find a linear line of the form  $y = mx + b$ , where  $y$  and  $x$  are variables,  $m$  is the slope, and  $b$  is the  $y$ -intercept. The formula to calculate slope  $m$  and the value of  $b$  is given by:

$$m = (n\sum xy - \sum y \sum x) / [n\sum x^2 - (\sum x)^2]$$

$$b = (\sum y - m\sum x) / n, \text{ Here, } n \text{ is the number of data points.}$$

Following are the steps to calculate the least square using the above formulas.

- Step 1: Draw a table with 4 columns where the first two columns are for  $x$  and  $y$  points.
- Step 2: In the next two columns, find  $xy$  and  $(x)^2$ .
- Step 3: Find  $\sum x$ ,  $\sum y$ ,  $\sum xy$ , and  $\sum (x)^2$ .
- Step 4: Find the value of slope  $m$  using the above formula.
- Step 5: Calculate the value of  $b$  using the above formula.
- Step 6: Substitute the value of  $m$  and  $b$  in the equation  $y = mx + b$

**Example 1:** Consider the set of points:  $(1, 1)$ ,  $(-2, -1)$ , and  $(3, 2)$ . Plot these points and the least-squares regression line in the same graph.

**Solution:** There are three points, so the value of  $n$  is 3

| $x$ | $y$ | $xy$ | $x^2$ |
|-----|-----|------|-------|
|-----|-----|------|-------|

|              |              |               |                 |
|--------------|--------------|---------------|-----------------|
| 1            | 1            | 1             | 1               |
| -2           | -1           | 2             | 4               |
| 3            | 2            | 6             | 9               |
| $\sum x = 2$ | $\sum y = 2$ | $\sum xy = 9$ | $\sum x^2 = 14$ |

Now, find the value of m, using the formula.

|                                                               |
|---------------------------------------------------------------|
| $m = (n\sum xy - \sum y \sum x) / [n\sum x^2 - (\sum x)^2]$   |
| $m = [(3 \times 9) - (2 \times 2)] / [(3 \times 14) - (2)^2]$ |
| $m = (27 - 4) / (42 - 4)$                                     |
| $m = 23/38$                                                   |

Now, find the value of b using the formula,

$$b = (\sum y - m \sum x) / n$$

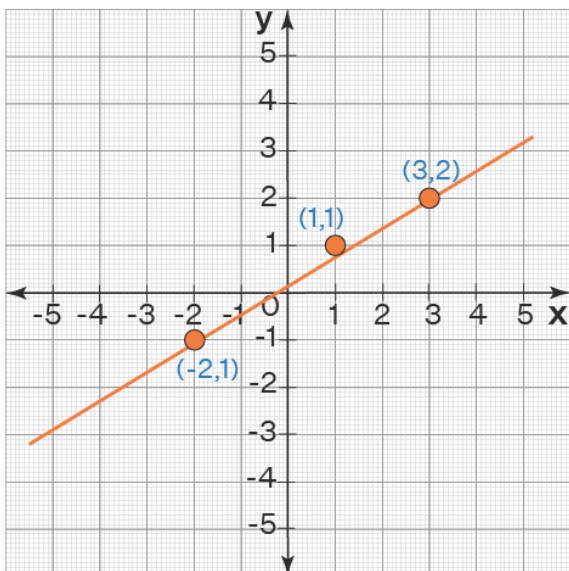
$$b = [2 - (23/38) \times 2] / 3$$

$$b = [2 - (23/19)] / 3$$

$$b = 15 / (3 \times 19)$$

$$b = 5/19$$

So, the required equation of least squares is  $y = mx + b = 23/38x + 5/19$ . The required graph is shown as:



Therefore, the equation of regression line is  $y = 23/38x + 5/19$ .